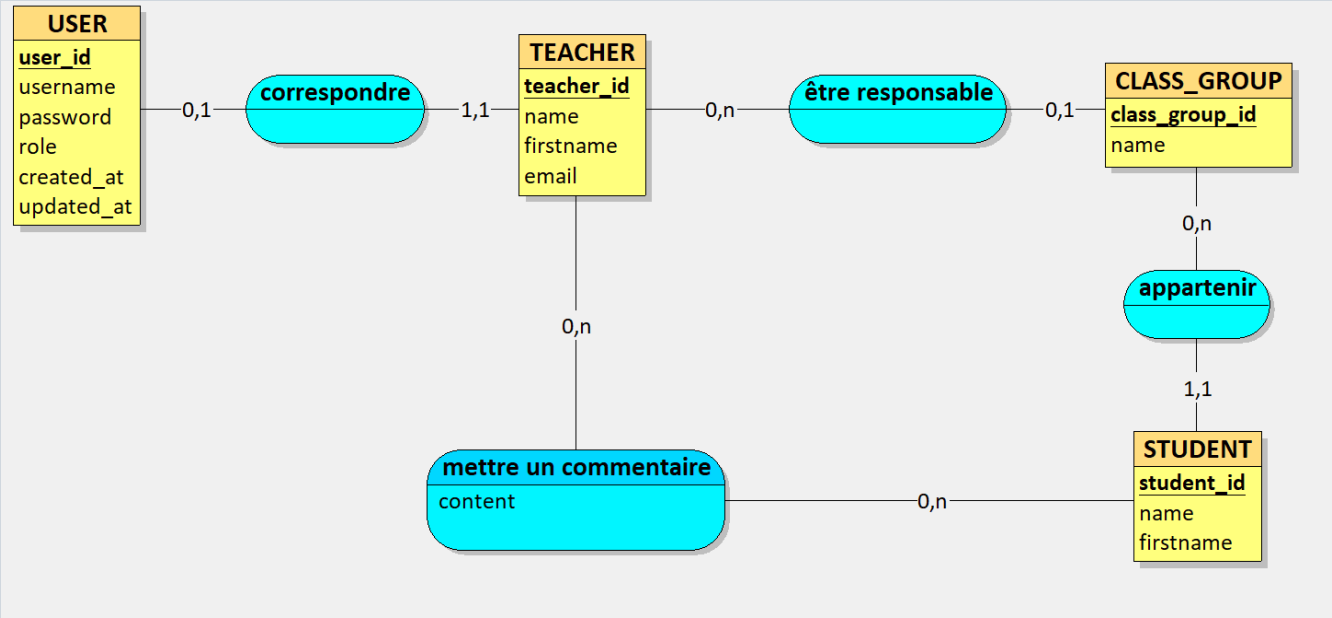
Backend pour le suivi des élèves - Step9

Nous allons maintenant nous intéresser aux relations entre les enseignants, les élèves et les classes.
Nous laissons volontairement les commentaires de côté pour l'instant ainsi que les utilisateurs.



Comme nous pouvons le voir :

- Un enseignant peut être responsable (Maître de classe) de plusieurs classes
- Une classe ne peut avoir qu'un responsable

On appelle cela une relation 1-N.

Idem pour la relation classe et élève.
C'est aussi une relation de type 1-N.

En base de données relationnelle, quand deux tables sont liées par une clé étrangère, on peut représenter cette relation dans les modèles Lucid pour simplifier les requêtes.

Lucid utilise deux décorateurs principaux pour ça :

`@hasMany` → l'entité courante possède plusieurs éléments de l'autre côté.

`@belongsTo` → l'entité courante appartient à un élément de l'autre côté.

Voyons maintenant concrètement comment intégrer cela.

Relation Enseignant - Classe

La migration

Avec AdonisJS 7, on commence toujours par la base de données.
Comme on travaille au niveau de la table MySQL, c'est simplement la clé étrangère `teacher_id` qui sera précisée.

Dans la migration des classes :

```
import { BaseSchema } from '@adonisjs/lucid/schema'

export default class extends BaseSchema {
  protected tableName = 'class_groups'

  async up() {
    this.schema.createTable(this.tableName, (table) => {
      table.increments('id')
      table.string('name').notNullable()

      // Relation : 1 classe → 1 enseignant
      table
        .integer('teacher_id') // on ajoute une colonne teacher_id correspondant au maître de classe
        .unsigned() // la valeur ne peut pas être négative
        .nullable() // une classe peut ne pas (encore) avoir d'enseignant responsable
        .references('id') // teacher_id fait référence à la colonne id d'une autre table. C'est ce lien qui définit la clé étrangère.
        .inTable('teachers') // Précise dans quelle table se trouve la colonne id référencée.

      table.timestamp('created_at')
      table.timestamp('updated_at')
    })
  }

  async down() {
    this.schema.dropTable(this.tableName)
  }
}
```

> 📝 **À noter**
> Après l'exécution des migrations, la classe `ClassGroupSchema` du fichier `database/schema.ts` contiendra automatiquement la propriété `teacherId` de type `number | null`.
> Le `| null` vient du `.nullable()` de la migration.

Les modèles

Les champs sont gérés par les classes générées dans `database/schema.ts`.
Dans les modèles, nous n'avons donc plus qu'à ajouter les **relations**.

Dans le modèle `Teacher` :

```
import { TeacherSchema } from '#database/schema'
import { hasMany } from '@adonisjs/lucid/orm'
import type { HasMany } from '@adonisjs/lucid/types/relations'
import ClassGroup from '#models/class_group'

export default class Teacher extends TeacherSchema {
  // Relation : 1 enseignant → N classes
  @hasMany(() => ClassGroup)
  declare classGroups: HasMany<typeof ClassGroup>
}
```

Dans le modèle `ClassGroup` :

```
import { ClassGroupSchema } from '#database/schema'
import { belongsTo } from '@adonisjs/lucid/orm'
import type { BelongsTo } from '@adonisjs/lucid/types/relations'
import Teacher from '#models/teacher'

export default class ClassGroup extends ClassGroupSchema {
  // Relation : 1 classe → 1 enseignant
  @belongsTo(() => Teacher)
  declare teacher: BelongsTo<typeof Teacher>
}
```

> 💡 **Question**
> Comment Lucid sait-il que la relation utilise la colonne `teacher_id` ?

Par convention, Lucid utilise le nom du modèle lié suivi de `Id` : `Teacher` → `teacherId` (soit la colonne `teacher_id` en base).
C'est pourquoi il est important de bien respecter les conventions de nommage.

Le seeder

Nous devons maintenant mettre à jour le seeder des classes :

```
// database/seeders/class_group_seeder.ts
import ClassGroup from '#models/class_group'
import Teacher from '#models/teacher'
import { BaseSeeder } from '@adonisjs/lucid/seeders'
```

```
export default class ClassGroupSeeder extends BaseSeeder {
  async run() {
    // Récupération des enseignants
    const teachers = await Teacher.all()

    await ClassGroup.createMany([
      { name: 'CIN1A', teacherId: teachers[0].id },
      { name: 'CIN1B', teacherId: teachers[0].id },
      { name: 'CIN1C', teacherId: teachers[1].id },
      { name: 'FID1', teacherId: teachers[1].id },
      { name: 'FID2', teacherId: teachers[2].id },
      { name: 'CID2A', teacherId: teachers[2].id },
      { name: 'CID2B', teacherId: teachers[3].id },
    ])
  }
}
```

Ordre des migrations et des seeders

Nous réinitialisons la db afin de prendre en compte les changements effectués.

```
node ace migration:fresh --seed
```

```
api-suivi-eleves-step-by-step (main)
$ node ace migration:fresh --seed
[ success ] Dropped tables successfully
[ info ] Upgrading migrations version from "1" to "2"
❯ migrated database/migrations/1746783669447_create_teachers_table
❯ migrated database/migrations/1746783673192_create_class_groups_table
❯ migrated database/migrations/1750316304397_create_users_table
❯ migrated database/migrations/1750316304401_create_access_tokens_table
❯ migrated database/migrations/1750316809384_create_students_table

Migrated in 1.36 s
❯ error     database/seeders/class_group_seeder
  Cannot read properties of undefined (reading 'id')
❯ completed database/seeders/student_seeder
❯ completed database/seeders/teacher_seeder
```

> ⚠️ QUESTION
> Pourquoi cette erreur ?

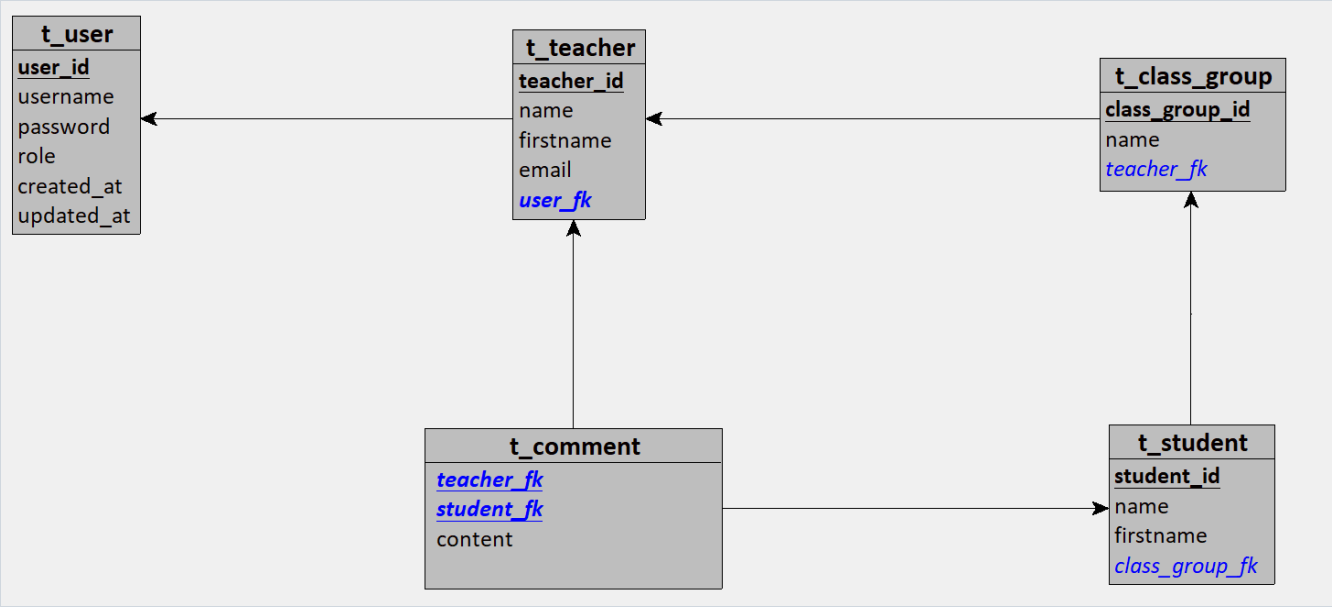
Maintenant que nous mettons en place les relations entre les tables, un ordre de création des tables et des seeders doit être défini.
En effet, je ne peux, dans notre cas, pas créer de classe si je n'ai pas d'enseignants.
Je dois donc créer les enseignants, puis les classes.

Or :

- les migrations sont exécutées dans l'ordre de leur nom de fichier, c'est-à-dire dans l'ordre du **timestamp** (donc dans l'ordre où nous les avons créées)
- les seeders sont exécutés dans l'**ordre alphabétique** : `class_group_seeder` est exécuté avant `teacher_seeder`

Nous allons donc définir un ordre logique dans la création des tables et des seeders.

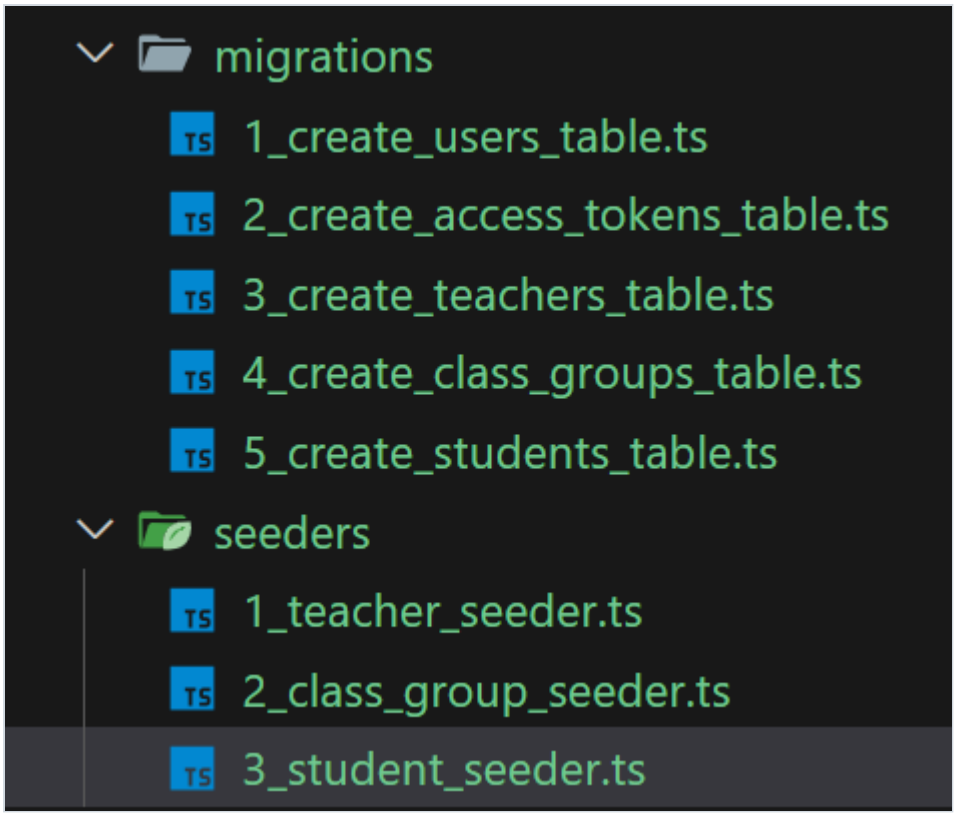
Prenons le MLD pour avoir une vision claire des clés étrangères :



Grâce au MLD, il est assez évident que nous devons respecter l'ordre suivant :

1. t_user (et la table `auth_access_tokens` qui en dépend)
2. t_teacher
3. t_class_group
4. t_student
5. t_comment

Respectons cet ordre en remplaçant le timestamp des migrations par un numéro et en préfixant les noms des seeders :



> 📝 **À noter**
> Les migrations `create_users_table` et `create_access_tokens_table` ont été créées par le kit starter. Il faut aussi les renommer.
> Grâce à l'option `naturalSort: true` de `config/database.ts`, le fichier `10_...` sera bien exécuté après le fichier `9_...`.

La commande ci-dessous devrait maintenant passer sans problème :

```
node ace migration:fresh --seed
```

```
$ node ace migration:fresh --seed
[ success ] Dropped tables successfully
[ info ] Upgrading migrations version from "1" to "2"
❯ migrated database/migrations/1_create_users_table
❯ migrated database/migrations/2_create_access_tokens_table
❯ migrated database/migrations/3_create_teachers_table
❯ migrated database/migrations/4_create_class_groups_table
❯ migrated database/migrations/5_create_students_table

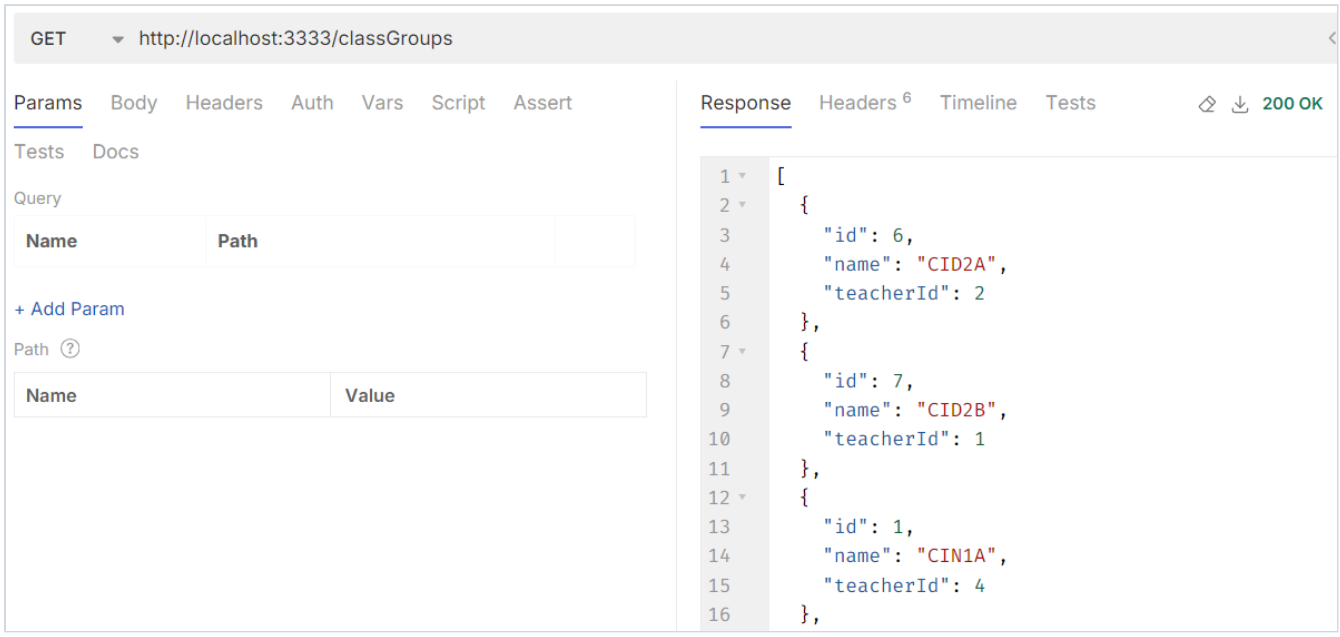
Migrated in 1.71 s
❯ completed database/seeders/1_teacher_seeder
❯ completed database/seeders/2_class_group_seeder
❯ completed database/seeders/3_student_seeder
```

Les contrôleurs

Route GET /classGroups

Maintenant, si nous testons à nouveau notre API, nous ne pouvons pas être complètement satisfaits.

Par exemple :



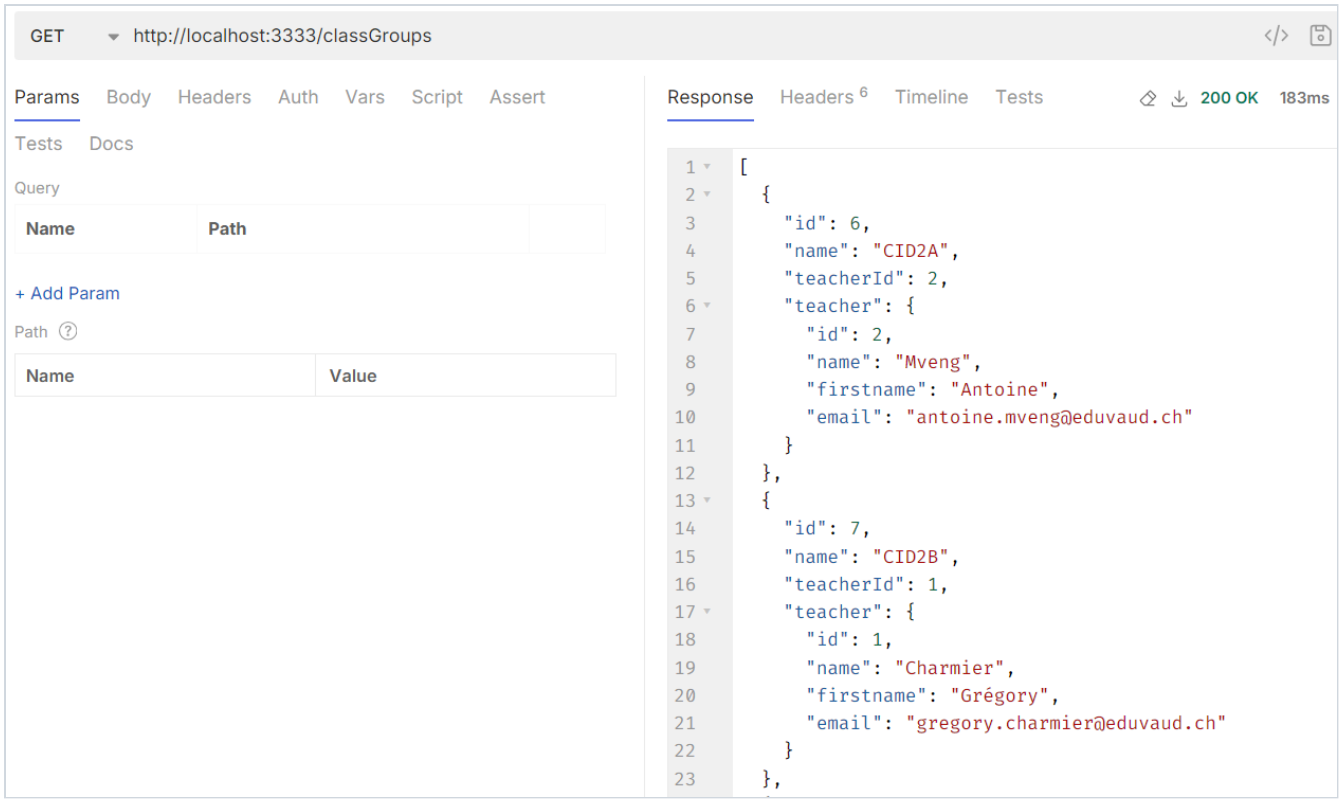
Nous aimerions bien avoir les informations du maître de classe dans le json des classes.

> 💡 QUESTION :
> Comment faire cela ?

Avec adonis, c'est super simple !
Il suffit de précharger les données grâce aux relations que l'on a définies dans le modèle.

En d'autres termes, il suffit de faire un `preload('teacher')` lors de la construction de la requête.

```
const classGroups = await ClassGroup.query().preload('teacher').orderBy('name', 'asc')
```



Route GET /classGroups/1

Pour la méthode show, nous devons un peu modifier la construction de la requête :

```
// Récupération d'une classe par son ID
const classGroup = await ClassGroup.query().preload('teacher').where('id', params.id).firstOrFail()
```

Route POST /classGroups/

Mise à jour du validateur pour ajouter le champ optionnel `teacherId` :

```
import vine from '@vinejs/vine'

export const classGroupValidator = vine.create({
  name: vine.string().minLength(2).maxLength(255),
  teacherId: vine.number().optional(),
})
```

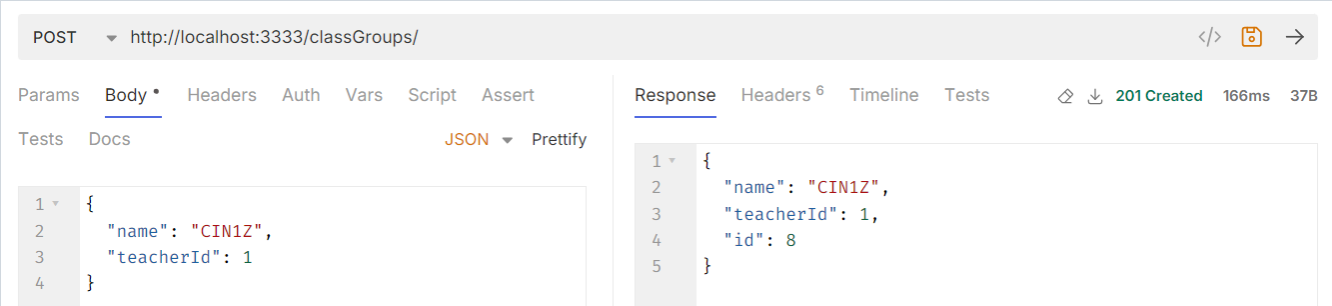
Pour le contrôleur, il y a très peu de chose à faire :

```
  /**
   * Créer une nouvelle classe
   */
  async store({ request, response }: HttpContext) {
    // Récupération des données envoyées par le client et validation des données
    const { name, teacherId } = await request.validateUsing(classGroupValidator)
```

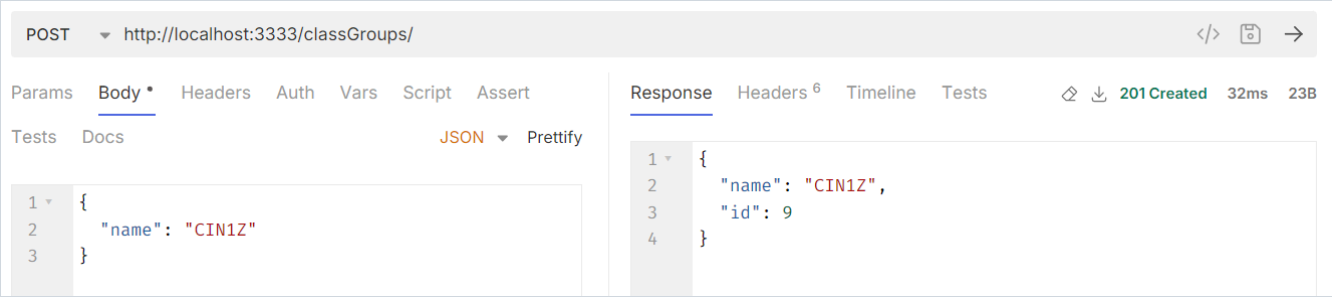
```
    // Création d'une nouvelle classe avec les données validées
    const classGroup = await ClassGroup.create({ name, teacherId })

    // On utilise `response.created` pour retourner un code HTTP 201 avec les données de la classe créée
    return response.created(classGroup)
  }
```

On teste l'ajout d'une classe avec un maître de classe :



On teste l'ajout d'une classe sans maître de classe :



Route PUT /classGroups/1

> 💪 EXERCICE : A vous de mettre à jour le code pour l'update.

Route DELETE /classGroups/1

Cette route n'est pas impactée par l'ajout de `teacherId`.

Conclusion

Nous mettrons en place la relation Classe - Elève dans la prochaine étape.